

Twitter Sentiment Analysis

Final Report on Findings

Analysing tweets to understand positive and negative feelings online

Author	Ballem Dileep
Dataset	Sentiment140 — 1.6 million tweets from Twitter
Method	NLTK text cleaning, TF-IDF, and 3 machine learning models
Models Tested	Naive Bayes, Logistic Regression, Support Vector Machine
Best Model	Logistic Regression — 75.33% accuracy, AUC 0.83
GitHub	https://github.com/dileepLE02/Twitter-sentiment-analysis
Date	May 2026

1. Introduction

Social media platforms like Twitter have millions of posts every day. Some of these posts contain hate speech and harmful language. It is too slow and costly for people to read and check every post by hand. This is why we need computers to help do this work automatically.

This project uses a method called Natural Language Processing (NLP) to teach a computer how to read tweets and decide if they are positive or negative. Although this project focuses on positive and negative feelings, the same method can be used to spot harmful language online.

What this project set out to do:

- Study a large set of tweets and understand what the data looks like.
- Clean the tweet text so it is ready for a computer to process.
- Train three different models and compare how well they work.
- Find the best model and explain why it was chosen.
- Share ideas on how this work can be improved in the future.

2. The Dataset

The dataset used in this project is called Sentiment140. It was collected from Twitter and contains 1.6 million tweets. Each tweet has a label that says whether it is positive or negative. These labels were set automatically based on the emoji symbols in the tweets. The emoji symbols were then removed from the tweet text.

What each column in the dataset contains:

Column	Name	Description
0	target	0 = Negative tweet, 4 = Positive tweet
1	id	A unique number for each tweet
2	date	The date and time the tweet was posted
3	flag	The search word used to find the tweet
4	user	The Twitter username of the person who posted
5	text	The actual tweet content — this is what we analyse

Data Quality Check Results:

- Total size: 1,599,999 rows and 6 columns.
- Missing data: zero — every row has all its information.
- Duplicate rows: zero — no tweet appears more than once.
- Class balance: exactly 50% positive and 50% negative tweets.
- Conclusion: the dataset is clean and ready to use without any fixes needed.

3. How We Cleaned the Tweet Text

Raw tweets contain a lot of noise — website links, usernames, numbers, and symbols that do not help the model understand the meaning. We used NLTK, a popular Python library, to clean the tweets step by step.

Step	Action	What It Does	Example
1	Lowercase	Change all letters to small letters	Hello World -> hello world
2	Remove links	Delete http and www addresses	http://t.co/xyz -> (removed)
3	Remove mentions	Delete @username tags and # symbols	@johns#fun -> (removed)
4	Remove symbols	Delete punctuation marks	can't wait!!! -> cant wait
5	Remove numbers	Delete digits	top 10 songs -> top songs
6	Split into words	Break the sentence into single words	i love this -> [i, love, this]
7	Remove stop words	Delete common words with little meaning	[i, love, this] -> [love]
8	Stemming	Cut words down to their root form	running -> run
9	Lemmatising	Change words to their dictionary form	better -> good

After cleaning, the tweet text is much simpler. For example, the tweet: 'I'm off too bed. Gotta wake up hella early tomorrow morning!' becomes: 'bed got wake hella earli tomorrow morn'.

4. Converting Text to Numbers

Machine learning models cannot read words directly. We need to convert the cleaned text into numbers. We used a method called TF-IDF, which stands for Term Frequency — Inverse Document Frequency.

How TF-IDF works:

- Term Frequency (TF): how often a word appears in one tweet.
- Inverse Document Frequency (IDF): how rare or common a word is across all tweets.
- A word that appears often in one tweet but rarely in others gets a high score.
- Common words like 'the' or 'and' get low scores because they appear everywhere.
- We kept the top 5,000 most useful words for our models.

Train and Test Split:

Because the full dataset is very large, we used a sample of 50,000 tweets. We split these into two groups: 40,000 tweets for training the model (80%) and 10,000 tweets for testing how well the model works (20%). The test tweets were never seen by the model during training.

5. The Three Models We Tested

Naive Bayes (Bernoulli)

This model works by looking at whether a word is present or not in a tweet. It then uses simple probability to decide if the tweet is positive or negative. It is very fast to train and works well as a starting point.

Result: Accuracy = 74.54% | AUC Score = 0.82

Logistic Regression

This model looks at the importance of each word and calculates a probability that the tweet belongs to a class. It is very good at handling text data that has been converted to numbers. It is also easy to understand and explain.

Result: Accuracy = 75.33% | AUC Score = 0.83

Support Vector Machine (SVM)

This model tries to draw the best possible line between positive and negative tweets in the number space. It is powerful for text classification but takes a bit more time to train than the other two.

Result: Accuracy = 74.22% | AUC Score = 0.82

6. Results

Accuracy and AUC Scores for All Models:

Model	Accuracy	AUC Score	Correct (Neg)	Correct (Pos)	Errors (Total)
Logistic Regression	75.33%	0.83	3,690	3,843	2,467
Naive Bayes (BNB)	74.54%	0.82	3,647	3,807	2,546
SVM (LinearSVC)	74.22%	0.82	3,651	3,771	2,578

Logistic Regression made the fewest total errors (2,467) and scored the highest on both accuracy (75.33%) and AUC (0.83). This makes it the clear winner and our chosen final model.

What the Confusion Matrix Tells Us:

A confusion matrix breaks down a model's results into four boxes. Here is what each box means for the Logistic Regression model:

Box	Count	Meaning
Correct Negative	3,690	Model said negative — it was negative (correct)
Correct Positive	3,843	Model said positive — it was positive (correct)
Wrong Positive	1,287	Model said positive — it was actually negative
Wrong Negative	1,180	Model said negative — it was actually positive

What the ROC-AUC Score Tells Us:

The AUC score measures how well the model can separate positive tweets from negative ones at any setting. A score of 1.0 is perfect. A score of 0.5 means the model is just guessing. All three models scored between 0.82 and 0.83, which means they are genuinely learning the difference between positive and negative tweets — not just guessing.

7. What Went Well and What Can Be Improved

What Went Well:

- The dataset was clean — no missing values or duplicate rows were found.
- The class balance was perfect — no extra work was needed to fix the data.
- The text cleaning pipeline removed noise effectively using NLTK.
- All three models gave consistent results, confirming the pipeline is reliable.
- Charts and word clouds clearly showed the difference between positive and negative tweet language.

Limitations and Weaknesses:

- TF-IDF does not understand word meaning — it only counts words. So it cannot detect sarcasm, jokes, or hidden meanings.
- We only used 50,000 tweets out of 1.6 million. Using more data would likely improve accuracy.
- The data was collected in 2009. New words, slang, and emojis are not included.
- The model only classifies two classes: positive and negative. There is no middle ground for neutral or unclear tweets.
- Hate speech is more complex than just negative feelings. More specific training data would be needed for that task.

Fair Use and Ethics:

Automated systems that flag content must be used carefully. If a model gets things wrong too often, it can incorrectly remove posts from people who did nothing wrong. Different cultures use words differently, and a model trained on one type of English may not work well for all users. A human reviewer should always be part of the final decision.

8. Ideas for Future Work

Use Better Word Representations

TF-IDF only looks at word frequency. Better models like Word2Vec or BERT understand the meaning of words and how they relate to each other. Using these would likely improve accuracy by a large amount.

Train on the Full Dataset

We used 50,000 tweets. Training on all 1.6 million tweets would give the model more examples to learn from and should improve accuracy by 2 to 5%.

Add More Sentiment Classes

Instead of just positive and negative, we could add: neutral, very positive, and very negative. This gives a clearer picture of how people feel.

Test on New Data

The model was trained on 2009 data. Testing it on recent tweets would show how well it handles new words and modern language patterns.

Build a Live System

The trained model could be connected to a live Twitter feed so that tweets are labelled in real time as they are posted.

Add Explanations

Tools like LIME or SHAP can show why the model made a decision. This makes the system easier to trust and check.

9. Conclusion

This project showed that it is possible to teach a computer to read tweets and decide if they are positive or negative with a reasonable level of accuracy. We built a full pipeline that goes from raw tweet data all the way to a working model.

Among the three models tested, Logistic Regression performed the best. It scored 75.33% accuracy and an AUC of 0.83. It also made the fewest mistakes overall. This model is simple, fast, and easy to explain — good qualities for a real-world system.

The project also showed that the quality of the text cleaning step matters a lot. Good cleaning with NLTK helped all three models perform better. Future improvements should focus on using smarter word representations like BERT, training on more data, and testing on recent tweets.

Final Choice: Logistic Regression Accuracy: 75.33% | AUC: 0.83 | Errors: 2,467 out of 10,000 test tweets Best suited for: social media monitoring, customer feedback analysis, content moderation support.

10. References

- [1] NLTK Project — Natural Language Toolkit Documentation. www.nltk.org
- [2] Scikit-learn — Machine Learning in Python. Pedregosa et al., JMLR 2011. www.scikit-learn.org
- [3] Go, Bhayani, Huang (2009) — Twitter Sentiment Classification using Distant Supervision. Stanford.
- [4] Python Software Foundation — Python Language Reference. www.docs.python.org
- [5] Sentiment140 Dataset — sourced from Twitter API, Stanford University.
- [6] Hunter, J.D. (2007) — Matplotlib: A 2D Graphics Environment. Computing in Science and Engineering.